



FORMATION INFORMATIQUE - SUITE

HTML et CSS - La suite

Semantique, cascade, variables, Grid,
images et a11y : la suite apres les bases.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. [Chapitre 1 - Rappel rapide et carte du livre](#)

- [Ce que tu gardes en tete](#)
- [La carte du livre](#)
- [Un fil rouge](#)
- [Ce dont tu as besoin](#)
- [Comment lire ce livre](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

2. [Chapitre 2 - HTML semantique : donner un sens a la page](#)

- [Pourquoi ca compte vraiment](#)
- [Les balises de structure que tu vas utiliser](#)
- [Une page blog, version semantique](#)
- [Landing simple](#)
- [Carte produit](#)
- [Ce que ce n'est pas](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

3. [Chapitre 3 - Cascade et specificite : qui gagne ?](#)

- [L'ordre compte](#)
- [Qu'est-ce qui pese plus ?](#)
- [Classes plutot que ID pour le style](#)
- [Selecteurs composes](#)
- [Heritage \(rapide\)](#)
- [Conflit classique sur une carte](#)
- [!important, le extincteur](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

4. [Chapitre 4 - Variables CSS : une couleur, un endroit](#)

- [Declarer et utiliser](#)
- [Pourquoi c'est genial sur une landing](#)
- [Valeur de secours](#)
- [Changer localement](#)
- [Noms qui aident](#)
- [Variables et formulaires](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

5. Chapitre 5 - CSS Grid : lignes et colonnes

- [Allumer la grille](#)
- [Colonnes plus expressives](#)
- [minmax et auto-fit \(idee simple\)](#)
- [Lignes et hauteur](#)
- [Gap, le petit heros](#)
- [Placement simple](#)
- [Alignement dans les cellules](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

6. Chapitre 6 - Mise en page avec Grid : header, contenu, aside

- [Le dessin de zones](#)
- [Brancher le HTML](#)
- [Variante avec menu lateral](#)
- [Empiler sur petit ecran](#)
- [Contenu dans les zones](#)
- [Header interne en Flex](#)
- [Zones nommees et debug](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

7. Chapitre 7 - Flex vs Grid : quand choisir quoi

- [Choisis Flex quand...](#)
- [Choisis Grid quand...](#)
- [Les deux ensemble](#)
- [Cas limites utiles](#)
- [Signes que tu as choisi le mauvais outil](#)
- [Mini decision rapide](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

8. Chapitre 8 - Images modernes : taille, cadrage, srcset

- [Ne jamais deborder : max-width](#)
- [Object-fit : cadrer sans deformer](#)
- [Le HTML reste responsable du fond](#)
- [Idee srcset, en simple](#)
- [Formats et poids \(sans dogmatisme\)](#)
- [Image dans une grille](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

9. Chapitre 9 - Styler un formulaire proprement

- [HTML d'abord, toujours](#)
- [Une colonne claire](#)
- [Focus visible](#)
- [Bouton coherent](#)
- [Deux champs cote a cote \(desktop\)](#)
- [Etats d'erreur \(leger\)](#)
- [Case a cocher et radio](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

10. [Chapitre 10 - Transitions legeres : du mouvement utile](#)

- [L'idee de base](#)
- [Que transitionner ?](#)
- [Durees raisonnables](#)
- [Liens et focus](#)
- [Plusieurs proprietes](#)
- [Respecter ceux qui veulent moins de mouvement](#)
- [Ce qu'on ne fait pas ici](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

11. [Chapitre 11 - Accessibilite suite : focus, contrastes, labels](#)

- [Focus visible : savoir ou on est](#)
- [Contrastes : lisible, pas juste joli](#)
- [Labels : dire le nom des champs](#)
- [Ordre de tabulation naturel](#)
- [Images et boutons](#)
- [Landing : checklist express](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

12. [Chapitre 12 - Mini-projet : page d'accueil responsive \(Grid + variables\)](#)

- [Brief](#)
- [Etape 1 - HTML semantique](#)
- [Etape 2 - Variables](#)
- [Etape 3 - Coquille et header](#)
- [Etape 4 - Hero](#)
- [Etape 5 - Grille de cartes](#)
- [Etape 6 - Formulaire et footer](#)
- [Etape 7 - Transitions legeres](#)
- [Etape 8 - Accessibilite express](#)
- [Criteres de reussite](#)
- [Erreur classique](#)
- [En vrai](#)

- [A toi](#)

13. [Chapitre 13 - A retenir](#)

- [Structure](#)
- [Cascade](#)
- [Variables](#)
- [Grid](#)
- [Flex vs Grid](#)
- [Images](#)
- [Formulaires](#)
- [Mouvement](#)
- [Accessibilite](#)
- [Mini-projet](#)
- [Fil rouge](#)
- [A toi](#)

14. [Chapitre 14 - Atelier Grid : blog en zones](#)

- [Brief](#)
- [Etapas](#)
- [Amorce CSS](#)
- [Contenu minimum](#)
- [Criteres de reussite](#)
- [Piege a eviter](#)
- [Bonus](#)
- [A toi](#)

15. [Chapitre 15 - Atelier variables : une marque en `:root`](#)

- [Brief](#)
- [Etapas](#)
- [Amorce palette + bascule](#)
- [Pastilles](#)
- [Criteres de reussite](#)
- [Piege a eviter](#)
- [Bonus](#)
- [A toi](#)

16. [Chapitre 16 - Atelier page : assembler une mini landing](#)

- [Brief](#)
- [Etapas](#)
- [Amorce hero + tarifs](#)
- [Contenu minimum](#)
- [Criteres de reussite](#)
- [Piege a eviter](#)
- [Bonus](#)
- [A toi](#)

17. [Chapitre 17 - Dark mode avec variables](#)

- [Prefers-color-scheme](#)

- [Color-scheme \(detail utile\)](#)
- [Bascule manuelle \(classe\)](#)
- [Images et ombres](#)
- [Formulaires en sombre](#)
- [Contrastes : double check](#)
- [Landing : hero en sombre](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

18. [Chapitre 18 - Perf CSS legeres : aller vite sans obsession](#)

- [Moins de CSS inutile](#)
- [Images : le vrai levier](#)
- [Eviter les reflows dramatiques](#)
- [Fonts](#)
- [Critical CSS ? Pas obligatoire ici](#)
- [Mesurer un peu](#)
- [CSS qui aide la stabilite](#)
- [Variables et maintenance](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

19. [Chapitre 19 - Debug CSS : outils, outline, hypotheses](#)

- [L'inspecteur, ton meilleur ami](#)
- [Outline sur tout \(temporaire\)](#)
- [Hypothese claire](#)
- [Pieges frequents \(checklist\)](#)
- [Box model et width](#)
- [Debug Grid](#)
- [Debug Flex](#)
- [Debug focus / a11y](#)
- [Methode en 5 minutes](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

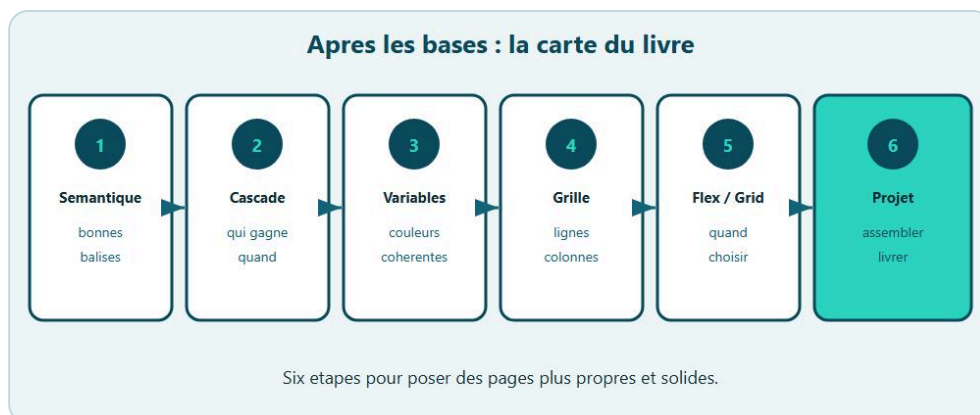
20. [Quiz final](#)

- [Questions](#)
- [Corriges](#)
- [Mini debrief](#)

21. [Bravo](#)

- [Ce que tu sais faire maintenant](#)
- [Petite mission \(si tu veux\)](#)
- [La suite, quand tu seras chaud](#)

Chapitre 1 - Rappel rapide et carte du livre



Après les bases : semantique, cascade, variables, Grid, a11y.

Tu as déjà fait le premier livre HTML et CSS. Tu sais structurer une page, écrire des balises, relier un fichier CSS, jouer avec les couleurs et les polices. Tu as vu les boîtes (margin, padding, border), Flexbox pour ranger les blocs, et une intro au responsive. Un peu d'accessibilité aussi.

Ici, on ne recommence pas à zéro. Pas de cours "c'est quoi une marge" pendant trois pages. Si Flexbox te semble encore flou, relis le premier livre. Ce tome-ci monte d'un cran : pages plus propres, plus solides, plus jolies sans tricher.

Chez DanielCraft, on aime une image simple. Les bases, c'était poser les murs et peindre une pièce. La suite, c'est poser un vrai plan d'étage, choisir une palette qui tient partout, faire bouger un bouton sans sursaut, et penser à ceux qui naviguent autrement que toi.

Ce que tu gardes en tête

Tu sais écrire un HTML clair avec `h1`, `p`, `a`, `img`. Tu sais cibler avec des classes. Tu sais aligner avec Flexbox. Tu as déjà ouvert la page sur téléphone (ou réduit la fenêtre). C'est assez pour avancer.

On ne revient pas sur "comment faire un lien" ni sur "display: flex" de A à Z. On s'en sert, oui. On ne le réexplique pas comme à un débutant total. Pareil pour margin et padding : tu les connais, on les utilise sans leçon débutant.

La carte du livre

Imagine une page produit, un petit blog, ou une landing pour un atelier. Pour que ça tienne vraiment, il te manque encore quelques pièces. Voici la carte, dans l'ordre où on va les poser.

D'abord le HTML sémantique : dire au navigateur "ça, c'est le menu", "ça, c'est l'article", "ça, c'est le pied de page". Ensuite la cascade et la spécificité : comprendre pourquoi une règle gagne sur une autre, sans paniquer. Puis les variables CSS : une couleur déclarée une fois, réutilisée partout.

Grid arrive ensuite : poser la page en lignes et colonnes, pas seulement en rangée Flexbox. On fera les bases, puis une vraie mise en page (header, contenu, aside). Juste après, Flex vs Grid : savoir choisir l'outil, pas empiler les deux au hasard.

Les images modernes : taille, `object-fit`, une idee simple de `srcset`. Les formulaires styles proprement. Des transitions legeres (hover doux, pas de cirque). L'accessibilite suite : focus visible, contrastes, labels. Un mini-projet qui assemble grid et variables. Un recap. Trois ateliers pour faire, pas seulement lire. Le mode sombre avec les variables. Un peu de perf CSS. Du debug (outils, outline, hypotheses). Un quiz. Et un bravo final.

Un fil rouge

On va souvent parler de trois exemples. Une page produit (carte, prix, bouton). Un blog (en-tete, articles, pied). Une landing simple (hero, sections, appel a l'action). Ces exemples reviennent, pour que tu sentes le progres.

Tu verras le meme geste sous plusieurs angles : structurer, thematiser, poser la grille, soigner le detail, verifier que ca reste utilisable au clavier.

Ce dont tu as besoin

Un editeur (VS Code, Cursor, ou autre). Un navigateur moderne. Les outils developpeur (F12). Pas de framework. Pas de Bootstrap obligatoire. Ici, on reste en HTML et CSS "nus", clairs, dans le navigateur.

Comment lire ce livre

Lis dans l'ordre au debut. Semantique, cascade, variables, puis Grid. Flex vs Grid apres. Images, formulaires, transitions, accessibilite avant le mini-projet. Les ateliers sont la pour faire. Dark mode, perf et debug viennent solidifier. Le quiz verifie. Le dernier chapitre, c'est pour souffler.

Si tu connais deja un sujet, lis quand meme le chapitre vite : le ton et les exemples servent de rappel utile pour la suite.

Erreur classique

Croire que "je connais les bases" = "je sais faire une vraie page pro". Les bases sont le moteur. La suite, c'est le plan, la coherence, le confort, et le respect des visiteurs. Sans semantique, ta page est un tas de `div`. Sans variables, changer de couleur devient un cauchemar. Sans Grid (ou un plan clair), le layout casse des que la fenetre change. Sans accessibilite, tu excludes du monde sans le vouloir.

Autre piege : sauter directement au mini-projet. Les ateliers et les chapitres courts sur le debug existent pour une raison. Une page "jolie" qui casse au Tab ou qui melange trois verts differents n'est pas finie.

En vrai

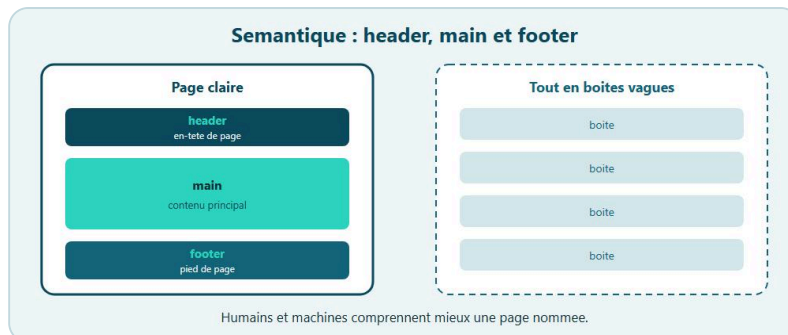
Ouvre une page que tu as deja codee (page perso, carte produit...). Note ce qui manque : un vrai `header` / `main` ? une couleur copiee quinze fois ? un layout qui casse en large ? un bouton sans etat focus ? un formulaire moche ? Ce livre repond a ca.

Si tu n'as aucune page sous la main, note trois pages web que tu aimes (boutique, blog, landing) et observe : ou est le menu, ou est le contenu, comment les cartes s'alignent, si le bouton change doucement au survol.

A toi

Ecris en trois phrases ce que tu veux construire a la fin. Pas un reseau social. Quelque chose de petit : "une page produit propre", "un blog a deux articles", "une landing pour mon atelier". Garde ce but. On y reviendra au mini-projet et aux ateliers.

Chapitre 2 - HTML sémantique : donner un sens à la page



Nommer les pièces : header, main, nav, article, footer.

Tu peux construire toute une page avec des `div`. Ça marche. Le navigateur affiche. Mais pour un humain qui lit le code, pour un lecteur d'écran, pour un moteur de recherche, une pile de `div` anonymes, c'est un mur sans portes ni pièces nommées.

Le HTML sémantique, c'est nommer les pièces. `header`, `nav`, `main`, `article`, `section`, `footer` (et d'autres). Tu dis ce que c'est, pas seulement "un bloc".

Chez DanielCraft, on traite ça comme le plan d'une boutique : vitrine, rayon, caisse, sortie. Si tout s'appelle "pièce", tu te perds. Si chaque zone a un rôle, la visite devient claire.

Pourquoi ça compte vraiment

Le navigateur n'a pas besoin de `header` pour afficher un bandeau. Une `div` avec une classe ferait l'affaire pour le CSS. Alors pourquoi s'embêter ?

Parce que le sens voyage. Un lecteur d'écran peut proposer "aller au contenu principal" grâce à `main`. Les titres dans un `article` ont un contexte. Les moteurs comprennent mieux la structure. Toi, dans six mois, tu rouvres le fichier et tu sais où est le menu sans jouer aux détectives.

Sur une landing, le hero n'est pas "div 1". C'est souvent un `header` ou une `section` claire dans le `main`. Sur un blog, chaque billet est un `article`. Sur une page produit, la fiche peut être un `article` ou une `section` bien titrée.

Les balises de structure que tu vas utiliser

header : en-tête d'une page ou d'une section. Logo, titre du site, parfois le menu. Une page peut avoir plusieurs `header` (un pour la page, un dans un article), mais commence simple : un en-tête de page.

nav : navigation principale (ou secondaire). Liens pour se déplacer dans le site. Pas besoin d'envelopper chaque lien isolé dans un `nav`. Réserve-le aux menus réels.

main : le contenu principal unique de la page. En général, un seul `main` par page. C'est "le cœur" : pas le menu global, pas le pied, pas la pub latérale.

article : un contenu qui tient tout seul (billet de blog, carte produit riche, commentaire autonome). Si tu peux l'imaginer extrait et encore compréhensible, `article` colle souvent.

section : un regroupement thématique avec souvent un titre. Sur une landing : section "Services", section "Témoignages", section "Contact". Ce n'est pas un `div` de plus : c'est un chapitre dans la page.

footer : pied de page ou pied d'un article. Mentions, liens secondaires, copyright. Comme **header**, il peut vivre a plusieurs niveaux, mais un pied de site suffit au debut.

aside : contenu a cote, complementaire (encart "A lire aussi", infos secondaires). Pas obligatoire partout. Utile des que tu as une vraie colonne laterale.

Une page blog, version semantique

```
<body>
  <header>
    <p class="logo">Mon Blog</p>
    <nav aria-label="Principale">
      <a href="/">Accueil</a>
      <a href="/articles">Articles</a>
    </nav>
  </header>

  <main>
    <article>
      <h1>Titre du billet</h1>
      <p>Intro du texte...</p>
    </article>
  </main>

  <aside>
    <h2>A lire aussi</h2>
    <ul>
      <li><a href="#">Autre billet</a></li>
    </ul>
  </aside>

  <footer>
    <p>Contact - Mentions</p>
  </footer>
</body>
```

Tu vois deja le plan. Le CSS viendra poser Grid ou Flex dessus. Le sens est la avant la peinture.

Landing simple

Sur une landing atelier, tu peux faire :

```
<header>
  <p class="marque">Atelier Ceramique</p>
  <nav>...</nav>
</header>
<main>
  <section class="hero">
    <h1>Apprends la terre en un week-end</h1>
    <p>Petit groupe, vrai four, vrai fun.</p>
    <a class="bouton" href="#inscription">Je m'inscris</a>
  </section>
  <section id="programme">
    <h2>Programme</h2>
    ...
  </section>
</main>
<footer>...</footer>
```

Le `h1` reste unique et fort. Les `h2` portent les sections. Tu ne sautes pas de `h1` a `h3` sans raison.

Carte produit

Une grille de cartes peut etre une liste d' `article` :

```
<main>
  <h1>Nos cafes</h1>
  <div class="grille">
    <article class="carte">
      <h2>Blend maison</h2>
      <p>Notes de cacao.</p>
      <p class="prix">9,90 €</p>
    </article>
  </div>
</main>
```

Le `div.grille` sert au layout CSS. Les `article` portent le sens de chaque produit.

Ce que ce n'est pas

Ce n'est pas une religion. Tu auras encore des `div` et des `span` pour le style ou les petits morceaux. L'idée : les zones importantes de la page ont un nom utile. Si tout est `section` ou tout est `article` sans reflechir, tu n'as gagne que du bruit.

`div` = boite neutre. Semantique = boite qui dit son role.

Erreur classique

Mettre trois `main` sur la meme page. Ou envelopper tout le body dans un seul `section` geant sans titre. Ou utiliser `header` uniquement parce que "ca fait pro", puis y coller le contenu principal entier.

Autre piege : confondre `nav` et une liste de liens sociaux dans le pied. Un groupe de liens peut rester une liste dans le `footer` sans `nav`, ou avec un `nav` clairement libelle si c'est vraiment de la navigation.

En vrai

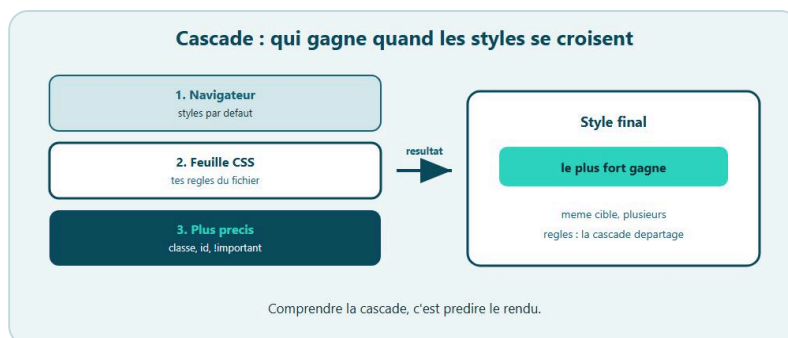
Prends une de tes pages. Remplace mentalement (puis dans le code) les gros `div` par `header`, `main`, `footer`. Ajoute `nav` autour du vrai menu. Si tu as un billet ou une fiche, tente `article`. Recharge. Visuellement, rien ne change si le CSS ciblait des classes. Structurellement, la page respire mieux.

Ouvre ensuite l'inspecteur : l'arbre DOM se lit comme un sommaire.

A toi

Recree une mini page "Produit" en HTML seul (presque pas de CSS). `header` avec nom de boutique + `nav`, `main` avec un `article` produit (titre, paragraphe, prix), `footer` avec une ligne de contact. Valide que tu as un seul `h1` et un seul `main`.

Chapitre 3 - Cascade et specificite : qui gagne ?



Cascade et specificite : qui gagne, sans paniquer.

Tu écris une règle. Tu en écris une autre. La couleur ne change pas. Ou elle change, mais pas celle que tu voulais. Bienvenue dans la cascade.

La cascade, ce n'est pas un monstre. C'est l'ordre dans lequel le navigateur décide quelle règle CSS l'emporte. La specificite, c'est le "poids" d'un sélecteur. Ensemble, elles expliquent presque tous les "pourquoi ça marche pas".

Chez DanielCraft, on compare ça à des couches de peinture : la dernière couche visible gagne, sauf si une couche précédente était beaucoup plus "forte" (vernis special). Tu n'as pas besoin de jargon opaque. Tu as besoin d'instinct.

L'ordre compte

À poids égal, la règle la plus basse dans le fichier (la dernière lue) gagne en général.

```
.titre {
  color: blue;
}

.titre {
  color: green;
}
```

Le titre sera vert. Deuxième déclaration, même sélecteur : elle efface la première.

Si tu charges deux feuilles CSS, l'ordre des `<link>` compte aussi. La feuille liée en dernier peut écraser la précédente, à specificite égale.

Qu'est-ce qui pèse plus ?

Sans rentrer dans une formule magique à retenir par cœur, retiens cette échelle simple, du plus faible au plus fort :

Un sélecteur de type (`p`, `h1`) pèse peu.

Une classe (`.carte`, `.bouton`) pèse plus.

Un identifiant (`#menu`) pèse encore plus.

Les styles inline dans le HTML (`style="..."`) pèsent très fort (à éviter pour le travail sérieux).

!important force encore plus fort (a utiliser presque jamais).

```
p {
  color: black;
}

.intro {
  color: #1a5f4a;
}

#accroche {
  color: red;
}
```

Sur `<p id="accroche" class="intro">`, le rouge de `#accroche` gagne. Pas parce qu'il est "mieux", parce qu'il pese plus.

Classes plutot que ID pour le style

Les ID sont utiles en HTML (cibles de liens, labels `for`). Pour le CSS du quotidien, prefere les classes. Pourquoi ? Parce qu'un ID trop present dans le CSS rend les overrides douloureux. Tu te retrouves a monter en **!important** ou a empiler des selecteurs monstrueux.

Sur une landing, `.hero-titre` se reutilise et se surcharge proprement. `#hero-titre` devient un piege des que tu veux une variante.

Selecteurs composes

`.carte .prix` est plus specifique qu'un simple `.prix`. Normal : tu as cible plus precisement.

```
.prix {
  color: #333;
}

.carte .prix {
  color: #1a5f4a;
}
```

Dans une carte produit, le prix prend le vert. Ailleurs, un `.prix` seul reste gris fonce.

Attention a ne pas ecrire des selecteurs kilometriques du type `body div main section article div p span`. Plus c'est long, plus c'est fragile.

Heritage (rapide)

Certaines proprietes se transmettent aux enfants (couleur de texte, police...). D'autres non (margin, padding, border en general). Si un paragraphe "prend" la couleur du `body`, ce n'est pas toujours la cascade qui ecrase : c'est l'heritage.

Quand tu debogues : demande-toi "est-ce une regle qui s'applique a cet element, ou une valeur heritee du parent ?"

Conflit classique sur une carte

```
a {  
  color: blue;  
}  
  
.carte a {  
  color: inherit;  
}  
  
.bouton {  
  color: white;  
  background: #1a5f4a;  
}
```

Si ton bouton est un lien `<a class="bouton">`, tu peux avoir une bataille entre `a`, `.carte a` et `.bouton`. Regarde dans l'inspecteur quelle règle est barrée. C'est la meilleure leçon.

!important, le extincteur

`!important` éteint l'incendie... et détruit souvent la maison ensuite. Une fois que tu en mets partout, plus rien n'est prévisible. Garde-le pour des cas rares (utile temporairement pour debug, ou contrainte très particulière). Le vrai fix, c'est : sélecteur un peu plus clair, ordre du fichier, moins d'ID.

Erreur classique

Penser "je vais mettre un ID, comme ça je suis sûr que ça gagne". Tu gagnes aujourd'hui, tu perds demain. Ou empiler trois classes inventées (`.bloc.special.urgent`) au lieu de clarifier la structure.

Autre piège : modifier le HTML en inline `style=""` parce que "ça marche tout de suite". Ça marche, et ça pollue.

En vrai

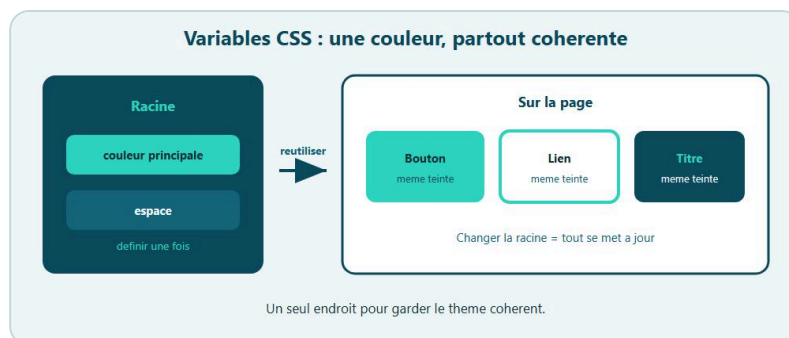
Ouvre une page où un style "refuse" de s'appliquer. F12, inspecte l'élément, onglet Styles. Cherche la propriété barrée. Remonte au sélecteur gagnant. Demande-toi : ordre ? poids ? inline ? Une fois que tu vois le gagnant, tu corriges avec intention : pas au hasard.

Fais le même exercice sur une landing avec menu + bouton. Regarde quelle règle colore les liens du menu vs le bouton CTA.

A toi

Crée une mini page avec un paragraphe `.intro` et un titre `#titre` (oui, un ID, pour l'exercice). Écris volontairement des règles conflictuelles (type, classe, ID) sur la couleur. Note sur papier qui gagne avant d'ouvrir l'inspecteur. Vérifie. Ensuite, retire l'ID du CSS et refais le style uniquement avec des classes. Garde la leçon.

Chapitre 4 - Variables CSS : une couleur, un endroit



Variables CSS : une couleur, un endroit.

Tu as deja change une couleur primaire dans dix classes differentes. Tu as oublie la onzieme. La page a l'air batarde. Ca arrive a tout le monde.

Les variables CSS (custom properties) resolvent ca. Tu declares une valeur une fois, souvent dans `:root`, puis tu la reutilises partout avec `var(...)`.

`:root`, c'est la racine du document. En pratique : "mes reglages globaux pour toute la page".

Chez DanielCraft, une palette claire, c'est une marque qui tient. Variables = palette centralisee.

Declarer et utiliser

```
:root {
  --couleur-principale: #1a5f4a;
  --couleur-fond: #f7f5f0;
  --couleur-texte: #222;
  --espace: 1rem;
}

body {
  background: var(--couleur-fond);
  color: var(--couleur-texte);
}

a {
  color: var(--couleur-principale);
}

.bouton {
  background: var(--couleur-principale);
  color: #fff;
  padding: var(--espace) calc(var(--espace) * 1.5);
}
```

Le double tiret `--` marque le nom de la variable. Tu choisis des noms clairs : `--couleur-principale` plutot que `--c1`.

Pourquoi c'est genial sur une landing

Imagine une landing avec bouton, liens, bordures d'accent, titre surligne. Si demain tu passes du vert au bleu marine, tu changes une ligne dans `:root`. Toute la page suit. C'est ca, une vraie palette.

Tu peux aussi stocker des tailles, des rayons, des ombres legeres :

```
:root {
  --rayon: 8px;
  --ombre: 0 4px 12px rgba(0, 0, 0, 0.08);
  --largeur-max: 1100px;
}

.carte {
  border-radius: var(--rayon);
  box-shadow: var(--ombre);
}

.wrap {
  max-width: var(--largeur-max);
  margin-inline: auto;
}
```

Sur un blog, le meme `--espace` rythme le padding des articles et le gap de la grille. Moins de magie, plus de coherence.

Valeur de secours

`var(--couleur-principale, #1a5f4a)` utilise la variable si elle existe, sinon le second argument. Utile quand tu experimentes ou quand une variable pourrait manquer.

```
.badge {
  background: var(--couleur-accent, #c45c26);
}
```

Changer localement

Une variable n'est pas obligatoirement globale. Tu peux la redefinir dans un bloc :

```
.carte-promo {
  --couleur-principale: #8b1e3f;
}

.carte-promo .bouton {
  background: var(--couleur-principale);
}
```

La carte promo a son accent. Le reste du site garde le vert. Les enfants de `.carte-promo` voient la nouvelle valeur. C'est puissant pour des themes de section sans dupliquer tout le CSS.

Noms qui aident

Mieux vaut `--couleur-principale` et `--couleur-fond` que `--vert` et `--beige` si tu comptes changer de teinte. Les noms de role (principale, fond, texte, bordure) survivent aux restyles. Les noms de teinte se periment.

Pour une boutique : `--prix`, `--dispo`, `--rupture` peuvent aussi etre des variables de couleur semantiques.

Variables et formulaires

```
:root {
  --champ-bordure: #ccc;
  --champ-focus: #1a5f4a;
}

input {
  border: 1px solid var(--champ-bordure);
}

input:focus {
  border-color: var(--champ-focus);
  outline-color: var(--champ-focus);
}
```

Tu prepares deja le terrain pour le chapitre formulaires et pour le dark mode plus tard.

Erreur classique

Declarer vingt variables "au cas ou" sans les utiliser. Ou copier la meme valeur en dur dans une classe "parce que c'est plus rapide" - tu reviens au probleme initial.

Autre piege : oublier que le nom est sensible a la casse et aux tirets. `--Couleur` n'est pas `--couleur`.

En vrai

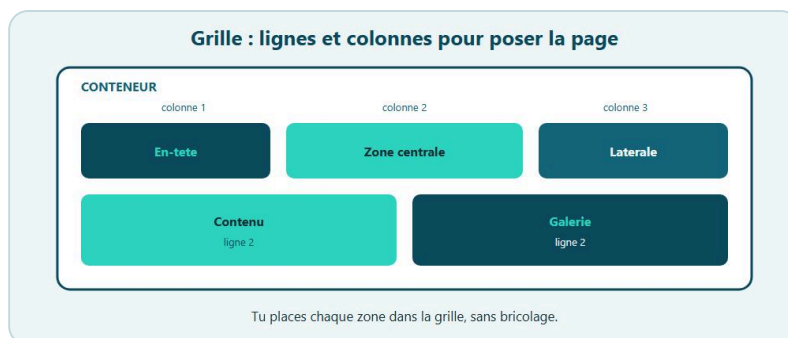
Ouvre une page existante. Liste les couleurs qui reviennent (boutons, liens, titres). Cree un `:root` avec trois a six variables. Remplace les valeurs en dur. Change une variable. Souris si toute la page suit.

Fais la meme chose pour `--espace` sur une grille de cartes produit : padding de carte et `gap` du parent.

A toi

Cree une mini landing : titre, paragraphe, bouton, une carte. Tout le theme passe par `:root` (`--couleur-principale`, `--fond`, `--texte`, `--rayon`, `--espace`). Change uniquement `:root` pour obtenir une variante "soir" (fond plus sombre, texte clair). Ne touche pas aux classes. Si ca marche, tu as compris les variables.

Chapitre 5 - CSS Grid : lignes et colonnes



Grid : lignes, colonnes, gap.

Flexbox, tu le connais : super pour une rangée, un menu, aligner dans un sens. Grid, c'est autre chose. Tu poses un vrai damier. Lignes et colonnes. Idéal pour une page, une galerie, un tableau de cartes produit.

Tu n'abandonnes pas Flexbox. Tu ajoutes un outil. Souvent : Grid pour la structure de page, Flexbox pour les petits alignements à l'intérieur d'une cellule.

Chez DanielCraft, Grid, c'est le plan au sol. Flex, c'est ranger les chaises dans une pièce.

Allumer la grille

```
.grille {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 1rem;
}
```

1fr veut dire "une part de l'espace libre". Trois colonnes égales, avec un espace **gap** entre les cases. Pas besoin de margin bridouilles entre chaque carte.

```
<div class="grille">
  <article class="carte">Produit A</article>
  <article class="carte">Produit B</article>
  <article class="carte">Produit C</article>
</div>
```

Sur une page boutique, ces trois cartes s'alignent proprement. Ajoute un quatrième produit : il passe à la ligne suivante, toujours dans la grille.

Colonnes plus expressives

Tu peux mélanger unités :

```
.page {
  display: grid;
  grid-template-columns: 200px 1fr;
  gap: 2rem;
}
```

Colonne fixe a gauche (menu), le reste a droite (contenu). Ou l'inverse pour une landing avec une grosse zone texte et une colonne étroite.

`repeat` évite de se répéter :

```
.galerie {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}
```

minmax et auto-fit (idée simple)

Pour une grille de cartes qui s'adapte sans media query compliquée :

```
.grille {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}
```

L'idée : chaque colonne fait au moins 220px, et on en met autant que ça rentre. Sur téléphone, une colonne. Sur grand écran, plusieurs. Tu peaufineras au chapitre mise en page et au mini-projet.

Lignes et hauteur

Souvent, les lignes se créent toutes seules selon le contenu. Tu peux aussi les guider :

```
.hero-grille {
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-template-rows: auto auto;
  gap: 1.5rem;
}
```

Pour commencer, concentre-toi sur les colonnes et le `gap`. Les lignes suivront.

Gap, le petit héros

`gap` (et `row-gap` / `column-gap` si besoin) espace les cellules sans casser les bords externes. Sur un blog en grille de billets, c'est net, prévisible, facile à retoucher avec une variable `--gap`.

```
:root {
  --gap: 1.25rem;
}

.grille {
  gap: var(--gap);
}
```

Placement simple

Sans dessiner toute la page, tu peux déjà étendre un élément :

```
.carte-large {  
  grid-column: 1 / -1;  
}
```

Cette carte occupe toute la largeur de la grille (de la première à la dernière ligne de colonnes). Utile pour un article épinglé au-dessus d'une grille de billets.

Placement par ligne/colonne :

```
.promo {  
  grid-column: 2;  
  grid-row: 1;  
}
```

Reste lisible. N'en abuse pas au début : une grille bien remplie dans l'ordre du HTML suffit souvent.

Alignement dans les cellules

`justify-items` et `align-items` influencent le contenu dans chaque case. `justify-content` / `align-content` influencent la grille dans son conteneur quand il reste de l'espace. Pour une galerie de cartes, commence par laisser les items s'étirer (`stretch` par défaut) et contrôle le padding dans `.carte`.

Erreur classique

Mettre `display: grid` et s'attendre à ce que Flexbox "merge" magiquement. Non : tu choisis l'outil du conteneur. Les enfants deviennent des items de grille.

Autre piège : trop de colonnes fixes en pixels sur mobile. Teste aussi en fenêtre étroite. Et `gap` énorme qui étouffe le contenu sur petit écran : passe `--gap` en plus petit sous une media query si besoin.

En vrai

Prends trois cartes produit (même fausses). Mets-les dans une grille à deux ou trois colonnes avec `gap`. Redimensionne la fenêtre. Observe comment les items se placent. Compare mentalement à un Flexbox avec `flex-wrap` : Grid pense déjà en colonnes déclarées.

Ajoute une quatrième carte "A la une" avec `grid-column: 1 / -1` au-dessus. Tu sens déjà le pouvoir du placement.

A toi

Crée une mini page "Ateliers" avec un titre et une grille de quatre cartes (titre + une phrase). `grid-template-columns: 1fr 1fr`, `gap: 1rem`. Sur feuille externe. Variables pour couleurs et gap. Pas besoin d'images. Le placement compte.

Chapitre 6 - Mise en page avec Grid : header, contenu, aside

Les bases de Grid, tu les as. Maintenant on pose une vraie page : en-tete, contenu, colonne laterale, pied. C'est le squelette d'un blog, d'une doc simple, d'une page "ressources" avec menu.

L'outil star ici : `grid-template-areas`. Tu dessines la page avec des mots. Ensuite chaque zone recoit son nom. C'est lisible. C'est modifiable. C'est parfait pour un PDF de formation comme chez DanielCraft : tu vois le plan avant le detail.

Le dessin de zones

```
.layout {
  display: grid;
  grid-template-columns: 2fr 1fr;
  grid-template-areas:
    "entete entete"
    "contenu aside"
    "pied pied";
  gap: 1.5rem;
  max-width: 1100px;
  margin-inline: auto;
  padding: 1rem;
}
```

Chaque ligne du dessin a le meme nombre de cellules. Ici : deux colonnes. L'entete prend les deux. Le contenu et l'aside se partagent la rangee du milieu. Le pied prend les deux.

Brancher le HTML

```
<div class="layout">
  <header class="entete">...</header>
  <main class="contenu">...</main>
  <aside class="aside">...</aside>
  <footer class="pied">...</footer>
</div>
```

```
.entete { grid-area: entete; }
.contenu { grid-area: contenu; }
.aside { grid-area: aside; }
.pied { grid-area: pied; }
```

Le nom dans `grid-area` doit matcher le mot du dessin. Si tu ecris `grid-area: header` mais que le dessin dit `entete`, ca ne se place pas.

Variante avec menu lateral

Sur grand ecran, un blog peut aussi avoir :

```
.layout {
  grid-template-columns: 180px 1fr 240px;
  grid-template-areas:
    "entete entete entete"
    "nav contenu aside"
    "pied pied pied";
}
```

Trois colonnes. Le `nav` a gauche, le contenu au centre, l'aside a droite. Sur une landing marketing, tu n'as peut-etre pas besoin d'aside : simplifie.

Empiler sur petit ecran

```
@media (max-width: 700px) {
  .layout {
    grid-template-columns: 1fr;
    grid-template-areas:
      "entete"
      "contenu"
      "aside"
      "pied";
  }
}
```

Une colonne. Ordre lisible : en-tete, contenu d'abord (priorite), puis aside, puis pied. Si tu as un `nav` separe, decide s'il passe juste sous l'entete ou apres le contenu. Pour un blog, menu haut puis articles reste naturel.

Contenu dans les zones

Dans `main.contenu`, tu peux encore avoir une grille interne pour les articles :

```
.liste-articles {
  display: grid;
  gap: 1rem;
}
```

Ou deux colonnes d'articles sur tablette. Grid imbriquee : normale et saine. La page a un Grid de structure ; une section a un Grid de cartes.

Dans l'aside : titre + liste de liens. Pas besoin d'en faire une usine.

Header interne en Flex

Souvent, le `header` en zone Grid utilise Flexbox a l'interieur : logo a gauche, liens a droite.

```
.entete {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
  flex-wrap: wrap;
}
```

Tu vois la complémentarité : Grid pose les pièces de la page, Flex range le contenu d'une pièce.

Zones nommées et debug

Si tu te perds, colore temporairement :

```
.entete { background: #e8f5ef; }  
.contenu { background: #fff8e8; }  
.aside { background: #eef0ff; }  
.pied { background: #f3f3f3; }
```

Tu vois immédiatement qui est où. Enlève les fonds après.

Erreur classique

Des lignes du dessin avec un nombre de cellules différent ("entete entete" puis "contenu" seul alors que tu as deux colonnes). Le navigateur rale ou se comporte bizarrement. Compte les mots.

Autre piège : oublier `grid-area` sur un enfant. Il se place quand même "dans le flux" de la grille et peut casser le dessin prévu.

Oublier le responsive : un aside étroit en pixels fixes à côté d'un contenu sur téléphone, ça étouffe. Repasse en une colonne.

En vrai

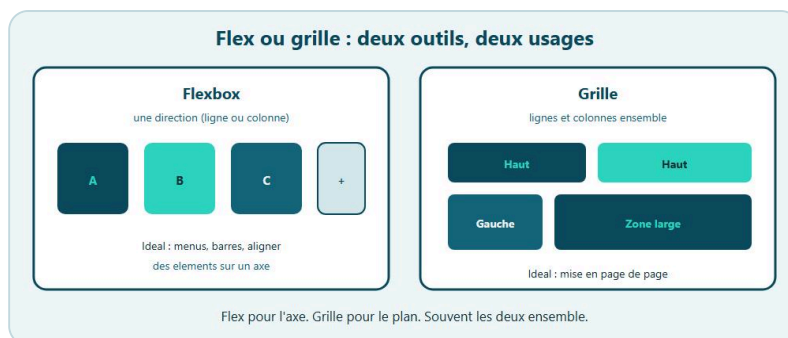
Construis le squelette d'un blog : deux articles dans le `main`, trois liens dans l' `aside`, un vrai `header` / `footer`. Areas sur desktop, pile sur mobile. Redimensionne lentement. Note à quelle largeur ça devient serré - c'est ton breakpoint.

Compare avec une ancienne page où tu avais tout fait en Flex + largeurs en %. Le dessin areas se lit souvent plus clairement.

A toi

Livre une page "Blog atelier" complète (HTML + CSS). Layout areas comme ci-dessus. Variables pour gap et couleurs. Sur `max-width: 700px`, une colonne. Critère : sans CSS, le HTML reste dans un ordre logique (entete, articles, aside, pied).

Chapitre 7 - Flex vs Grid : quand choisir quoi



Flex ou Grid : choisir le bon outil.

Tu as deux outils puissants. La question n'est plus "lequel est le meilleur ?" mais "lequel pour ce job ?".

Flexbox brille dans une dimension : une rangée ou une colonne. Aligner, distribuer l'espace, centrer un bouton, faire un menu, coller une icône à un label.

Grid brille en deux dimensions : lignes et colonnes en même temps. Page entière, galerie, tableau de cartes, zones nommées.

Chez DanielCraft, on dit souvent : Grid pour le plan de la page, Flex pour le mobilier dans chaque pièce. Ce n'est pas une loi absolue. C'est un excellent réflexe de départ.

Choisis Flex quand...

Tu ranges des éléments sur une seule ligne (ou une seule colonne) et tu veux contrôler l'espace entre eux.

Exemples concrets :

Un menu horizontal : logo + liens.

Un bandeau : titre à gauche, bouton à droite.

Une carte produit : image au-dessus (bloc), puis en bas une rangée prix + bouton en Flex.

Centrer un bloc dans un hero (souvent Flex sur le conteneur, ou Grid avec place-items - les deux marchent).

```
.menu {
  display: flex;
  gap: 1rem;
  align-items: center;
  flex-wrap: wrap;
}
```

Simple, direct, parfait.

Choisis Grid quand...

Tu penses déjà "colonnes" et "lignes". Tu veux qu'un élément occupe toute la largeur pendant que d'autres se partagent une rangée. Tu dessines header / contenu / aside.

```
.galerie {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));  
  gap: 1rem;  
}
```

Une galerie de cartes blog ou boutique : Grid. Essayer la meme chose en Flex + largeurs + wrap fonctionne, mais tu te bats plus pour des colonnes regulieres.

Les deux ensemble

C'est normal et recommande.

```
<div class="layout">  
  <header class="entete">...</header>  
  <main>  
    <div class="grille-cartes">...</div>  
  </main>  
</div>
```

`.layout` en Grid (areas). `.entete` en Flex (logo / nav). `.grille-cartes` en Grid (produits). Chaque conteneur choisit son outil. Les enfants ne "melangent" pas les modes du parent : c'est le parent qui decide.

Cas limites utiles

Une seule colonne de sections qui s'empilent ? Tu n'as pas toujours besoin de Grid. Le flux normal HTML + des marges peut suffire. N'active pas un moteur de layout pour rien.

Un formulaire de deux champs cote a cote sur desktop ? Flex ou Grid, les deux collent. Grid avec `1fr 1fr` est tres clair. Flex avec `flex: 1` aussi. Choisis celui que tu lis le mieux dans six mois.

Un tableau de donnees ? Le HTML `<table>` reste souvent le bon sens semantique. Grid peut mimer un tableau pour des cards, mais pour des vraies donnees tabulaires, garde le tableau.

Signes que tu as choisi le mauvais outil

Tu ajoutes des largeurs en % partout sur des enfants Flex pour "faire des colonnes" et tu pries pour le wrap : regarde Grid.

Tu utilises Grid puis tu te bats avec `align-items` pour un simple menu de liens : reviens a Flex.

Tu mets `position: absolute` partout pour sauver un layout : souvent un signe que Grid/Flex n'ont pas ete poses. (L'absolu a sa place pour un badge sur image, pas pour toute la page.)

Mini decision rapide

Rangee de controles, menu, barre d'actions → Flex.

Page ou galerie en damier → Grid.

Les deux niveaux (page + barre) → Grid dehors, Flex dedans (ou l'inverse selon le cas, mais reste coherent).

Erreur classique

Declarer `display: flex` et `display: grid` sur le meme element en esperant un hybride. La derniere declaration gagne. Un seul mode par conteneur.

Autre piege : tout convertir en Grid parce que "c'est moderne". Flex n'est pas obsolete. Il est specialise.

En vrai

Reprends une page landing. Identifie trois conteneurs. Pour chacun, ecris sur un papier "Flex" ou "Grid" et pourquoi. Puis code. Si un choix gene, switch et note la difference de lisibilite du CSS. L'outil qui produit le CSS le plus court et clair gagne souvent.

Sur une carte produit : essaye le corps de la carte en colonne Flex (`flex-direction: column`) et le footer de carte (prix + bouton) en Flex rangee. La page parente reste en Grid.

A toi

Construis une mini page "Boutique" : layout page en Grid (entete / grille / pied), entete en Flex, grille de 3 produits en Grid `auto-fit` . Aucun `absolute` pour le plan. Une phrase en commentaire CSS au-dessus de chaque conteneur : `/ Grid: plan /` ou `/ Flex: menu /` .

Chapitre 8 - Images modernes : taille, cadrage, srcset

Une belle image mal gérée casse une page. Elle débord. Elle écrase le texte. Elle pèse 4 Mo et fait attendre le téléphone en 4G. Les bases, tu les as (`img` , `alt`). Ici, on range le comportement CSS et on touche à l'idée de `srcset` .

Chez DanielCraft, une image de produit nette et légère, c'est du respect pour le visiteur. Pas besoin d'être ingénieur perf : quelques réflexes suffisent.

Ne jamais déborder : max-width

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

Règle d'or. L'image peut retrecir dans son conteneur. Elle garde ses proportions grâce à `height: auto` . Sans ça, une grande photo dans une colonne étroite fait scroller horizontalement. Moche.

Sur une carte blog, le conteneur a une largeur ; l'image suit.

Object-fit : cadrer sans déformer

Parfois tu fixes une hauteur de cadre (hero, vignette carrée). L'image source n'a pas le même ratio. `object-fit` décide comment remplir.

```
.carte img {  
  width: 100%;  
  height: 180px;  
  object-fit: cover;  
  display: block;  
}
```

`cover` : remplit le cadre, recadre ce qui dépasse. Idéal pour des vignettes produit uniformes.

`contain` : image entière visible, éventuelles bandes. Utile pour un logo ou un schéma qu'on ne doit pas couper.

`fill` : étire (souvent moche). Évite sauf cas spécial.

```
.hero-media img {  
  width: 100%;  
  height: 320px;  
  object-fit: cover;  
  object-position: center top;  
}
```

`object-position` dit où ancrer le cadrage (visage en haut, produit centre...).

Le HTML reste responsable du fond

```

```

`width` et `height` (attributs) aident le navigateur à réserver l'espace et limitent le saut de layout pendant le chargement. Le CSS peut ensuite redimensionner.

`alt` vide (`alt=""`) seulement si l'image est purement décorative. Sinon, décris ce que l'image apporte, pas "image1".

Idee srcset, en simple

Souvent tu as une grande photo. Sur téléphone, télécharger le monstre 2000px est du gaspillage. `srcset` propose plusieurs fichiers ; le navigateur choisit.

```

```

Lecture douce :

`srcset` liste les fichiers avec leur largeur réelle (`400w` = largeur 400px du fichier).

`sizes` dit approximativement quelle largeur l'image occupera à l'écran. Ici : plein écran sous 600px, sinon environ 400px (carte dans une grille).

`src` reste un fallback.

Tu n'as pas besoin de maîtriser tous les cas. Comprends l'idée : plusieurs poids, le navigateur choisit. Pour un projet perso, commencer par compresser une seule image correctement reste déjà un huge win.

`srcset` vient après.

Formats et poids (sans dogmatisme)

JPEG/WebP pour photos. PNG pour schémas à aplats / transparence. Évite d'envoyer un PNG énorme pour une photo. Comprime avant upload (outils en ligne, Squoosh, script...). Une carte produit à 200 Ko au lieu de 3 Mo, ça se sent.

Le CSS ne compresse pas le fichier. Il contrôle l'affichage. Le poids, c'est le fichier source.

Image dans une grille

```
.grille {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}

.carte img {
  width: 100%;
  aspect-ratio: 4 / 3;
  object-fit: cover;
}
```

`aspect-ratio` aide a garder des cartes alignees meme avant chargement complet. Combo sympathique avec `object-fit: cover`.

Erreur classique

Mettre une largeur fixe en pixels enorme sur mobile. Oublier `alt`. Utiliser `background-image` pour une image informative (moins accessible, plus penible a gerer pour le contenu). Forcer `height` sans `object-fit` et ecraser le sujet.

Autre piege : cinq images hero en slider des le premier projet. Une bonne image bien cadre bat un carrousel bancal.

En vrai

Prends une page produit. Applique `max-width: 100%` globalement sur `img`. Uniformise les vignettes avec hauteur fixe + `object-fit: cover`. Pese ton fichier image (proprietes du fichier). S'il depasse 500 Ko pour une vignette, recompresse.

Si tu as deux tailles de fichier, tente un petit `srcset` sur une seule image et regarde dans l'onglet Reseau du navigateur quelle variante part selon la largeur de fenetre.

A toi

Page "Galerie atelier" : grille de quatre images (meme fausses URLs ou placeholders). Toutes en `object-fit: cover` dans un cadre `aspect-ratio: 1 / 1`. Un `alt` utile chacune. Bonus : une des images en `srcset` simple a deux largeurs.

Chapitre 9 - Styler un formulaire proprement

Tu sais faire un formulaire HTML (champs, labels, bouton submit). La, on le rend agreable et clair. Pas un theme de science-fiction. Un formulaire ou on a envie de taper son email.

Sur une landing, le formulaire d'inscription est souvent le moment de verite. Sur un blog, le contact aussi. Chez DanielCraft, un champ lisible + un bouton evident, ca convertit mieux qu'une animation tape-a-l'oeil.

HTML d'abord, toujours

```
<form class="form" action="#" method="post">
  <p>
    <label for="nom">Nom</label>
    <input id="nom" name="nom" type="text" autocomplete="name" required>
  </p>
  <p>
    <label for="email">Email</label>
    <input id="email" name="email" type="email" autocomplete="email" required>
  </p>
  <p>
    <label for="message">Message</label>
    <textarea id="message" name="message" rows="4"></textarea>
  </p>
  <p>
    <button type="submit">Envoyer</button>
  </p>
</form>
```

Label visible, relie avec `for` / `id`. Pas seulement un placeholder. Le placeholder disparaît à la saisie ; le label reste. `autocomplete` aide les navigateurs et les gestionnaires de mots de passe.

Une colonne claire

```
.form {
  display: grid;
  gap: 1rem;
  max-width: 28rem;
}

.form label {
  display: block;
  margin-bottom: 0.35rem;
  font-weight: 600;
}

.form input,
.form textarea {
  width: 100%;
  padding: 0.65rem 0.75rem;
  border: 1px solid var(--champ-bordure, #ccc);
  border-radius: var(--rayon, 8px);
  font: inherit;
  background: #fff;
  color: inherit;
}
```

`font: inherit` évite les champs avec une police système bizarre différente du reste. `width: 100%` dans un conteneur borne, c'est confortable sur mobile.

Focus visible

Quand on tabule jusqu'au champ, il faut voir où on est.

```
.form input:focus,  
.form textarea:focus {  
  border-color: var(--couleur-principale, #1a5f4a);  
  outline: 2px solid var(--couleur-principale, #1a5f4a);  
  outline-offset: 2px;  
}
```

Ne supprime pas `outline` sans remplacement. C'est un classique d'accessibilité (on y revient au chapitre 11).

Bouton cohérent

```
.form button {  
  border: 0;  
  border-radius: var(--rayon, 8px);  
  padding: 0.75rem 1.25rem;  
  background: var(--couleur-principale, #1a5f4a);  
  color: #fff;  
  font: inherit;  
  cursor: pointer;  
}  
  
.form button:hover {  
  filter: brightness(1.05);  
}  
  
.form button:focus-visible {  
  outline: 2px solid var(--couleur-principale, #1a5f4a);  
  outline-offset: 3px;  
}
```

Le bouton ressemble au reste du site grâce aux variables. Pas un gris système perdu dans le coin.

Deux champs côte à côte (desktop)

```
.form-rangee {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 1rem;  
}  
  
@media (max-width: 600px) {  
  .form-rangee {  
    grid-template-columns: 1fr;  
  }  
}
```

Prenom / nom sur desktop, empilés sur mobile. Grid rend ça limpide.

Etats d'erreur (leger)

Sans JavaScript pousse, tu peux deja styler `:invalid` avec prudence (certains navigateurs le montrent tot) :

```
.form input:user-invalid {  
  border-color: #a12;  
}
```

Si `:user-invalid` n'est pas dispo partout chez toi, garde un style d'erreur via une classe `.champ-erreur` ajoutez plus tard. L'important : le message d'erreur en texte, pas seulement la couleur.

Case a cocher et radio

Laisse assez d'espace cliquable. Associe toujours label et input. Evite de reduire la case a 8px invisibles.

```
.form-check {  
  display: flex;  
  align-items: flex-start;  
  gap: 0.5rem;  
}
```

Flex pour aligner case + texte. Simple.

Erreur classique

Placeholder comme seul label. Champs trop etroits au centre avec du blanc partout sur mobile. Bouton submit sans type clair. Couleur de bordure trop pale sur fond pale (contraste). `outline: none` global sur tous les inputs.

Autre piege : styler uniquement l'etat "beau au repos" et oublier hover / focus / disabled.

En vrai

Reprends le formulaire contact d'une landing. Passe-le en grille verticale, labels au-dessus, variables de marque, focus visible. Navigue au clavier uniquement. Demande a quelqu'un de le remplir sans explication : s'il hesite, clarifie les labels.

Compare un champ natif non style et le tien : le tien doit rester evidemment un champ, pas un mystere design.

A toi

Formulaire "Inscription atelier" : nom, email, choix de creneau (`select`), message, bouton. Style coherent avec un `:root` de variables. Rangee prenom/nom en deux colonnes desktop. Test Tab complet. Aucun `outline: none` sans alternative.

Chapitre 10 - Transitions legeres : du mouvement utile

Un bouton qui change de couleur d'un coup, c'est correct. Un bouton qui glisse doucement vers sa nouvelle couleur, c'est plus agreable. Les transitions CSS font ca : interpoler entre deux etats.

On reste leger. Pas de cirque. Pas de page qui rebondit partout. Chez DanielCraft, le mouvement sert la clarte : "tu survoles, ca reagit". Rien de plus.

L'idee de base

Tu as un etat normal et un etat `:hover` (ou `:focus-visible`). Tu dis au navigateur : "quand ca change, prends 200ms".

```
.bouton {
  background: var(--couleur-principale);
  color: #fff;
  transition: background-color 200ms ease, transform 200ms ease;
}

.bouton:hover {
  background: #144a3a;
  transform: translateY(-1px);
}
```

Important : la propriete `transition` se place en general sur l'etat de base (`.bouton`), pas seulement dans `:hover` . Ainsi l'aller et le retour sont doux.

Que transitionner ?

De bons candidats : `color` , `background-color` , `opacity` , `transform` , `box-shadow` leger, `border-color` .

Des candidats plus risques : `width` , `height` , `top` , `left` (souvent moins fluides / plus coutueux).

Preferer `transform` et `opacity` quand tu peux.

```
.carte {
  transition: box-shadow 200ms ease, transform 200ms ease;
}

.carte:hover {
  transform: translateY(-2px);
  box-shadow: var(--ombre-hover, 0 8px 20px rgba(0, 0, 0, 0.12));
}
```

Sur une grille de produits, un leger lift au survol guide l'oeil sans hurler.

Durees raisonnables

150ms a 300ms pour de l'UI. Au-dela, ca trainasse. En dessous de 100ms, on ne sent presque rien.

`ease` , `ease-out` : naturels pour des hovers. Evite `linear` sur un bouton (souvent mecanique).

Liens et focus

```
a {
  color: var(--couleur-principale);
  transition: color 150ms ease;
}

a:hover {
  color: #0f3d30;
}

a:focus-visible {
  outline: 2px solid var(--couleur-principale);
  outline-offset: 2px;
}
```

La transition sur la couleur, oui. Mais le focus doit rester net : un outline clair, pas uniquement un fondu invisible.

Plusieurs proprietes

Tu peux lister plusieurs transitions, ou utiliser `transition: all 200ms ease` avec prudence. `all` est pratique en proto, un peu large en production : tu risques de transitionner des choses non voulues. Préférer la liste explicite quand tu peaufines.

Respecter ceux qui veulent moins de mouvement

Certains reglent leur systeme pour reduire les animations.

```
@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    transition-duration: 0.01ms !important;
    animation-duration: 0.01ms !important;
  }
}
```

Version simple et efficace pour un petit site. Le site reste utilisable, sans imposer de mouvement.

Ce qu'on ne fait pas ici

Pas de carousel auto agressif. Pas de titre qui rebondit en boucle. Pas d'animation permanente qui distrait la lecture d'un article. Les vrais `@keyframes` complexes, ce n'est pas le coeur de ce livre : une transition hover bien faite vaut mieux.

Erreur classique

Mettre `transition` seulement dans `:hover` : l'aller est doux, le retour est sec. Ou transitionner pendant 2 secondes : l'utilisateur a déjà cliqué ailleurs.

Autre piège : `transform` sur un élément qui casse un layout (ok en général) combine à des marges animées (moins ok). Garde simple.

En vrai

Sur une landing, ajoute une transition a : le bouton principal, les cartes, les liens du menu (couleur). Rien d'autre. Navigue a la souris, puis au clavier. Si ca semble calme et net, c'est gagne.

Enleve ensuite toutes les transitions et remets-les une par une. Tu sentiras celles qui apportent vraiment.

A toi

Page produit avec une carte et un bouton. Transitions 200ms sur carte (ombre + léger translate) et bouton (fond). Ajoute le media `prefers-reduced-motion`. Vérifie hover et focus-visible.

Chapitre 11 - Accessibilite suite : focus, contrastes, labels

Le premier livre a plante le decor : textes lisibles, alt, structure. Ici, on solidifie trois reflexes qui changent vraiment la vie des gens : focus visible, contrastes, labels (et un peu d'ordre au clavier).

Accessibilite, ce n'est pas un badge. C'est "est-ce que quelqu'un d'autre que moi peut utiliser cette page sans souffrir ?". Chez DanielCraft, on le traite comme de la qualite produit, pas comme un bonus cosmetique.

Focus visible : savoir ou on est

Beaucoup de gens naviguent au clavier (Tab, Shift+Tab, Entree). Si tu enlèves les contours de focus parce que "c'est moche", tu les perds dans la page.

```
:focus-visible {  
  outline: 2px solid var(--couleur-principale, #1a5f4a);  
  outline-offset: 2px;  
}
```

`:focus-visible` cible surtout le focus clavier (selon navigateurs / contexte), ce qui evite parfois un anneau au simple clic souris. Tu peux affiner par composant (boutons, liens, champs).

```
.bouton:focus-visible {  
  outline: 3px solid var(--couleur-principale);  
  outline-offset: 3px;  
}
```

Test minimal : charge ta page, cache la souris, Tab jusqu'au bout. Tu dois toujours voir clairement l'element actif.

Contrastes : lisible, pas juste joli

Texte gris pale sur fond beige, ca fait "design soft"... et illegible au soleil ou avec une vue fatiguee. Vise un contraste solide entre texte et fond.

Regles pratiques :

Texte courant sombre sur fond clair, ou texte clair sur fond sombre.

Les liens ne se distinguent pas seulement par la couleur (souligne au hover/focus, ou poids).

Les placeholders tres clairs ne remplacent pas un label.

Tu peux verifier avec un outil de contraste (extensions, sites de check). Si tu hesites entre deux beiges, choisis le plus lisible. La marque survit a un contraste correct.

```
:root {  
  --couleur-texte: #1b1b1b;  
  --couleur-fond: #f7f5f0;  
  --couleur-muette: #444;  
}
```

`--couleur-muette` trop claire (#aaa sur blanc) devient un piege pour les mentions legales.

Labels : dire le nom des champs

On l'a vu au chapitre formulaires. On insiste parce que c'est le bug n°1 des "jolis" formulaires.

```
<label for="email">Email</label>
<input id="email" type="email" name="email">
```

Pas ça :

```
<input type="email" placeholder="Email">
```

Le placeholder peut aider en exemple (nom@domaine.fr), pas en remplacement du label.

Pour une case a cocher :

```
<label>
  <input type="checkbox" name="newsletter">
  J'accepte de recevoir la newsletter
</label>
```

Ou `for` / `id` separees. L'important : le nom est annonce clairement.

Ordre de tabulation naturel

L'ordre du HTML guide le Tab. Si tu deplaces visuellement des blocs avec Grid/Flex, l'ordre visuel peut diverger de l'ordre clavier. En cas de doute, rearrange le HTML pour coller a l'ordre de lecture logique (entete, contenu, aside...).

Evite `tabindex` positifs inventes (1, 2, 3...) : ca devient un labyrinthe. `tabindex="0"` parfois pour rendre un element focusable ; `tabindex="-1"` pour du focus programme. Au debut, le HTML bien ordonne suffit.

Images et boutons

Image informative → `alt` utile.

Bouton icone seul → nom accessible (texte visible, ou `aria-label` si vraiment icone seule).

```
<button type="button" aria-label="Fermer">X</button>
```

Mieux encore : un texte "Fermer" visible si tu as la place.

Landing : checklist express

Un seul `h1`.

Menu atteignable au clavier.

CTA avec focus visible.

Contraste du texte hero (attention texte blanc sur photo claire : ajoute un voile sombre).

Formulaire avec labels.

```
.hero {  
  background: linear-gradient(rgba(0, 0, 0, 0.45), rgba(0, 0, 0, 0.45)),  
    url("hero.jpg") center/cover;  
  color: #fff;  
}
```

Le degrade aide le contraste du texte sur image.

Erreur classique

`outline: none` global. Liens oranges sur fond orange. Formulaire "flottant" sans labels. Modale ou menu qui piege le focus (sujet avance : au moins, ne casse pas Tab sur une page simple).

Autre piege : croire que "personne ne navigue au clavier". Faux. Et meme parmi les souris, certains ont besoin de focus clair.

En vrai

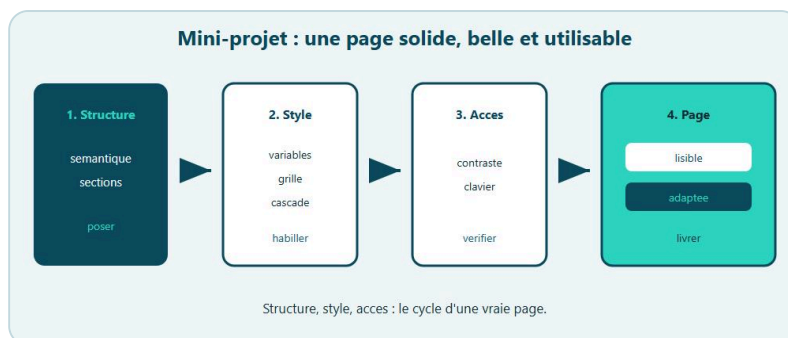
Prends ta page produit ou landing. Fais le parcours Tab complet. Note chaque trou (focus invisible, ordre bizarre). Monte le contraste d'un texte secondaire trop pale. Verifie chaque `input` pour un label.

Si tu peux, active un lecteur d'ecran basique juste cinq minutes (Narrateur sur Windows, VoiceOver sur Mac) et ecoute la page. Tu entendras vite les trous.

A toi

Ameliore une page existante avec trois commits mentaux : (1) `:focus-visible` global propre, (2) correction de deux contrastes faibles, (3) labels complets sur un formulaire. Ecris en trois lignes ce que tu as change. C'est ton "rapport a11y" personnel.

Chapitre 12 - Mini-projet : page d'accueil responsive (Grid + variables)



Mini-projet : page d'accueil responsive.

On assemble. Tu vas construire une petite page d'accueil : marque, menu, hero, grille de cartes, bandeau d'appel, formulaire court, pied. Responsive. Theme via variables. Layout via Grid. Pas de framework.

L'objectif n'est pas la perfection pixel. C'est une page coherente, lisible, utilisable au clavier, qui tient sur telephone et desktop. Le genre de base qu'on aimerait livrer pour un atelier, une boutique naive, ou une vitrine DanielCraft miniature.

Brief

Sujet libre, mais concret. Exemples : "Atelier ceramique", "Cafe de quartier", "Coach sportif", "Librairie".

La page contient :

Un header avec nom + navigation (2-4 liens).

Un hero avec `h1`, une phrase, un bouton.

Une section "Nos pepites" (ou equivalent) : grille de 3 ou 4 cartes (titre, texte, lien).

Une section contact : petit formulaire (nom, email, message).

Un footer simple.

Sur grand ecran : hero confortable, grille multi-colonnes. Sur petit ecran : tout s'empile, boutons aisees a taper.

Etape 1 - HTML semantique

Pose le squelette sans te battre avec le style.

```

<body>
  <header class="site-header">...</header>
  <main>
    <section class="hero">...</section>
    <section class="section" id="cartes">...</section>
    <section class="section" id="contact">...</section>
  </main>
  <footer class="site-footer">...</footer>
</body>
  
```

Un `h1` dans le hero. Des `h2` de section. Cartes en `article`. Formulaire avec labels. Liens du menu qui pointent vers `#cartes` et `#contact` si tu veux du one-page.

Etape 2 - Variables

```
:root {
  --couleur-principale: #1a5f4a;
  --couleur-fond: #f7f5f0;
  --couleur-texte: #1b1b1b;
  --couleur-carte: #fff;
  --rayon: 10px;
  --espace: 1rem;
  --gap: 1.25rem;
  --largeur-max: 1100px;
  --ombre: 0 4px 14px rgba(0, 0, 0, 0.08);
}
```

Branche `body`, liens, boutons, cartes sur ces variables. Aucune couleur magique éparpillée si tu peux l'éviter.

Etape 3 - Coquille et header

```
body {
  margin: 0;
  font-family: Georgia, "Times New Roman", serif;
  background: var(--couleur-fond);
  color: var(--couleur-texte);
  line-height: 1.5;
}

.wrap {
  max-width: var(--largeur-max);
  margin-inline: auto;
  padding: var(--espace);
}

.site-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: var(--espace);
  flex-wrap: wrap;
}
```

Flex pour le header. Serif volontaire pour sortir des stacks par défaut, tout en restant simple. Tu peux choisir une autre police web si tu veux, tant que c'est lisible.

Etape 4 - Hero

```
.hero {
  display: grid;
  gap: var(--espace);
  padding: calc(var(--espace) * 2) 0;
}

.hero h1 {
  font-size: clamp(1.8rem, 4vw, 3rem);
  line-height: 1.15;
  margin: 0;
}
```

`clamp` donne une taille fluide entre min et max. Pas obligatoire, mais agreable. Un seul CTA principal dans le hero.

Etape 5 - Grille de cartes

```
.grille {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: var(--gap);
}

.carte {
  background: var(--couleur-carte);
  border-radius: var(--rayon);
  padding: var(--espace);
  box-shadow: var(--ombre);
}
```

Images optionnelles : si tu en mets, `max-width: 100%` + `object-fit: cover` + `aspect-ratio`.

Etape 6 - Formulaire et footer

Reprend les patterns du chapitre 9. Formulaire en grid verticale, max-width raisonnable. Footer discret avec contraste correct.

Etape 7 - Transitions legeres

Boutons et cartes : 200ms. `prefers-reduced-motion` en bas de fichier.

Etape 8 - Accessibilite express

Tab complet. Focus visible. Labels. Contraste hero. `alt` si images.

Criteres de reussite

La page se lit sans CSS (ordre logique).

Changer 2 variables change le theme de facon visible.

La grille passe d'une a plusieurs colonnes sans casser.

Pas de scroll horizontal sur mobile.

Focus visible sur liens, boutons, champs.

Erreur classique

Tout faire en `position: absolute`. Copier un theme enormement complexe. Oublier le formulaire labels. Mettre cinq CTA aussi forts les uns que les autres dans le hero. Grille fixe `1fr 1fr 1fr` sans media query qui ecrase le telephone.

En vrai

Chronometre-toi 90 minutes. Livrable imparfait > concept parfait non code. Ensuite tu peaufines 30 minutes : contrastes, gaps, textes.

Montre la page a quelqu'un. Demande : "C'est pour quoi, en cinq secondes ?" Si la reponse matche ton intention, le hero fait son job.

A toi

Livre `index.html` + `styles.css`. Nom du projet dans le header. Trois cartes minimum. Un formulaire. Un theme 100 % variables. Bonus : une variante en commentant une autre palette dans `:root` (tu basculeras vraiment au chapitre dark mode).

Chapitre 13 - A retenir

Tu as parcouru pas mal de terrain. Avant les ateliers et les chapitres "affutage", on pose les pieces sur la table. Pas une encyclopedie. Une checklist mentale.

Structure

Le HTML semantique nomme les pieces : `header`, `nav`, `main`, `article`, `section`, `footer`, parfois `aside`. Les `div` restent pour le layout neutre. Un `main`, un `h1` fort.

Cascade

A poids egal, la derniere regle gagne. Les classes pesent plus que les types. Les ID pesent encore plus (evite-les pour le style quotidien). `!important` presque jamais. L'inspecteur montre qui gagne.

Variables

`:root` pour la palette et les rythmes (`--couleur-principale`, `--espace`, `--rayon`...). `var(--nom)` partout. Une variable peut se redefinir localement dans un bloc. Changer le theme = changer le `:root` d'abord.

Grid

`display: grid`, `grid-template-columns`, `gap`, `repeat`, `minmax` / `auto-fit` pour des galleries. `grid-template-areas` pour une page header/contenu/aside/pied. Placement simple avec `grid-column` si besoin.

Flex vs Grid

Flex : une dimension (menus, barres, alignements). Grid : deux dimensions (plan de page, damier). Souvent Grid dehors, Flex dedans. Un mode par conteneur.

Images

`max-width: 100%` + `height: auto`. `object-fit: cover` pour des cadres. `alt` utile. Poids du fichier soigne. `srcset` comme idee pour servir la bonne largeur.

Formulaires

Labels visibles relies. Champs en pleine largeur dans un conteneur borne. Focus clair. Bouton de marque. Deux colonnes en Grid sur desktop si utile, une sur mobile.

Mouvement

Transitions courtes sur l'etat de base. Hover doux. `prefers-reduced-motion` respecte. Pas de cirque.

Accessibilite

Focus visible, contrastes, labels, ordre Tab logique. Voile sombre si texte sur photo. Tester au clavier.

Mini-projet

Une home responsive qui assemble variables + Grid + semantique + formulaire + transitions legeres, c'est le niveau vise.

Fil rouge

Page produit, blog, landing : les memes reflexes. Chez DanielCraft, on prefere une page simple solide a une page chargee fragile.

A toi

Sans regarder tes notes, ecris dix lignes : cinq choses que tu sais faire maintenant, cinq pieges que tu veux eviter. Garde ce papier a cote des ateliers.

Chapitre 14 - Atelier Grid : blog en zones

Atelier pratique. Objectif : construire le squelette d'un petit blog en Grid avec `grid-template-areas`, puis l'adapter en une colonne sur petit ecran.

Pas de framework. HTML + CSS. Tu peux reutiliser des couleurs via des variables.

Brief

La page contient un en-tete (titre du blog + menu), un contenu principal avec deux articles, une colonne "A lire aussi" avec trois liens, un pied. Sur grand ecran : contenu et aside cote a cote, entete et pied pleine largeur. Sur petit ecran : tout s'empile dans un ordre lisible (en-tete, contenu, aside, pied).

Theme libre : "Notes de voyage", "Cuisine du dimanche", "Journal d'atelier"...

Etapas

1. Ecris le HTML semantique : `header`, `nav`, `main` avec deux `article`, `aside`, `footer`.
2. Cree un conteneur `.layout` en `display: grid`.
3. Dessine les areas sur grand ecran (entete pleine largeur, contenu + aside, pied pleine largeur).
4. Relie chaque piece avec `grid-area`.
5. Ajoute `gap` et une largeur max centree (`max-width` + `margin-inline: auto`).
6. Declare un `:root` minimal (couleurs, gap).
7. Sous `max-width: 700px` (ou similaire), repasse en une colonne et redefinis `grid-template-areas` pour empiler.
8. Style léger des articles (fond carte, rayon) sans casser le plan.

Amorce CSS

```
:root {
  --couleur-principale: #1a5f4a;
  --fond: #f7f5f0;
  --carte: #fff;
  --gap: 1.5rem;
}

.layout {
  display: grid;
  grid-template-columns: 2fr 1fr;
  grid-template-areas:
    "entete entete"
    "contenu aside"
    "pied pied";
  gap: var(--gap);
  max-width: 1000px;
  margin-inline: auto;
  padding: 1rem;
}

.entete { grid-area: entete; }
.contenu { grid-area: contenu; }
.aside { grid-area: aside; }
.pied { grid-area: pied; }
```

```
@media (max-width: 700px) {  
  .layout {  
    grid-template-columns: 1fr;  
    grid-template-areas:  
      "entete"  
      "contenu"  
      "aside"  
      "pied";  
  }  
}
```

Adapte si tu ajoutes une colonne menu. L'important : le dessin reste coherent (meme nombre de cellules par ligne).

Contenu minimum

Chaque article : un `h2`, un paragraphe, un lien "Lire la suite". L'aside : un titre et une liste de liens. Pas besoin de vrais articles longs. Un `h1` dans l'entete ou au debut du `main` (un seul).

Criteres de reussite

Les zones sont au bon endroit. Le HTML se lit sans CSS dans la tete (ordre logique). Sur mobile, rien ne debord. Tu n'as pas triche avec des `absolute` pour le plan general. Les variables pilotent au moins fond et gap.

Piege a eviter

Copier un layout Grid trouve sur le web sans comprendre les noms de zones. Si tu bloques, colorie temporairement chaque area (fond different) pour voir qui est ou.

Autre piege : mettre l'aside avant le `main` dans le HTML pour "aider" le CSS desktop, puis se retrouver avec l'aside avant les articles au Tab sur mobile. Preferer l'ordre de lecture : contenu puis aside, et laisser Grid placer.

Bonus

Une carte "article epingle" avec `grid-column: 1 / -1` au-dessus des deux articles dans le `main` (petite grille interne). Ou un header en Flex (titre / nav).

A toi

Livre la page. Change le texte des articles. Si le layout tient en large et en etroit, l'atelier est valide. Montre-la a quelqu'un : demande ou est "le menu" et "l'article" en trois secondes.

Chapitre 15 - Atelier variables : une marque en `:root`

Atelier pratique. Objectif : thématiser une petite page produit uniquement via des variables CSS. Tu dois pouvoir changer toute l'ambiance en touchant `:root`, sans chasser des couleurs en dur dans cinquante classes.

Brief

Page "Carte produit" : nom de boutique, une carte (titre produit, court texte, prix, bouton), trois pastilles d'info en dessous (livraison, retour, origine). Tu livres deux palettes : une claire, une "soir" (plus sombre). La bascule peut être manuelle : tu commentes/decommentes un bloc `:root`, ou tu ajoutes une classe sur `body` (encore mieux).

Etapes

1. Pose le HTML sémantique minimal (`header`, `main`, `article`, `footer`).
2. Crée un `:root` avec au moins : `--couleur-principale`, `--fond`, `--texte`, `--carte`, `--rayon`, `--espace`, `--ombre`.
3. Branche `body`, carte, liens, bouton, pastilles sur `var(...)`.
4. Zéro couleur hex "en dur" hors de `:root` (sauf éventuel blanc/noir de texte sur bouton si tu les declares aussi en variables).
5. Crée une seconde palette.
6. Fais basculer via `body.theme-soir` (recommande) ou en échangeant le bloc `:root`.
7. Vérifie les contrastes dans les deux thèmes.
8. Ajoute une transition légère sur le bouton (toujours via variables pour la couleur).

Amorce palette + bascule

```
:root {
  --couleur-principale: #1a5f4a;
  --fond: #f7f5f0;
  --texte: #1b1b1b;
  --carte: #ffffff;
  --rayon: 10px;
  --espace: 1rem;
  --ombre: 0 4px 14px rgba(0, 0, 0, 0.08);
}

body.theme-soir {
  --couleur-principale: #5dcaa5;
  --fond: #14221c;
  --texte: #eef5f1;
  --carte: #1c3229;
  --ombre: 0 4px 14px rgba(0, 0, 0, 0.4);
}

body {
  background: var(--fond);
  color: var(--texte);
}

.carte {
  background: var(--carte);
```

```

border-radius: var(--rayon);
padding: calc(var(--espace) * 1.25);
box-shadow: var(--ombre);
}

.bouton {
background: var(--couleur-principale);
color: var(--fond);
border: 0;
border-radius: var(--rayon);
padding: 0.7rem 1.1rem;
transition: filter 200ms ease;
}

```

Pour tester le theme soir : `<body class="theme-soir">`. Tu verras au chapitre dark mode une version plus "systeme". Ici, l'important est la discipline des variables.

Pastilles

Une petite grille `repeat(3, 1fr)` sur desktop, une colonne sur mobile. Chaque pastille utilise `--carte` et une bordure `color-mix` ou une bordure simple en `var(--couleur-principale)`.

```

.infos {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(140px, 1fr));
gap: var(--espace);
}

```

Criteres de reussite

Changer uniquement les variables change fond, texte, carte, bouton.

Les deux themes restent lisibles (contraste).

Pas de chasse au hex dans les classes de composants.

Le HTML reste simple et semantique.

Piege a eviter

Garder `#1a5f4a` dans `.bouton` "temporairement". Le temporaire reste. Autre piege : theme sombre avec texte `#666` sur fond `#14221c` (illisible). Monte le contraste.

Bonus

Ajoute `--font-titre` et `--font-corps` en variables (noms de familles). Ou un `--accent` pour le prix uniquement.

A toi

Livre la page + les deux themes. Fais une capture mentale (ou réelle) des deux versions. Si tu peux basculer en une classe, tu es pret pour le chapitre dark mode.

Chapitre 16 - Atelier page : assembler une mini landing

Atelier pratique. Objectif : produire une landing one-page complete, propre, responsive, en collant sémantique + variables + Grid + formulaire + transitions légères + focus visible.

C'est le cousin guide du mini-projet. Si tu as déjà une home du chapitre 12, tu peux la refondre ici avec un brief plus strict. Sinon, pars de zéro.

Brief

Landing pour un atelier d'une journée (sujet libre : photo, pain, couture, code chez DanielCraft...).

Sections obligatoires :

1. 1. Header (marque + nav ancres)
2. 2. Hero (promesse + CTA)
3. 3. Programme (3 étapes en grille)
4. 4. Tarifs (2 cartes côte à côte sur desktop)
5. 5. Inscription (formulaire)
6. 6. Footer

Contraintes : pas de framework, pas d'`absolute` pour le plan général, thème 100 % variables, Tab impeccable.

Étapes

1. 1. Écris tout le HTML d'un trait, textes provisoires OK.
2. 2. Pose `:root` (palette + espace + rayon + largeur max).
3. 3. Style le header en Flex.
4. 4. Hero en Grid simple (texte ; optionnellement une image à côté sur desktop).
5. 5. Section programme en `auto-fit` / `minmax`.
6. 6. Section tarifs en `1fr 1fr` puis `1fr` sous 600px.
7. 7. Formulaire labels + focus (chapitre 9).
8. 8. Transitions 200ms sur CTA et cartes.
9. 9. Media `prefers-reduced-motion`.
10. 10. Passe le parcours Tab et corrige.
11. 11. Teste à 320px, 768px, 1200px de large (ou redimensionne à la main).

Amorce hero + tarifs

```
.hero {
  display: grid;
  gap: 1.5rem;
  align-items: center;
}

@media (min-width: 800px) {
  .hero {
    grid-template-columns: 1.2fr 1fr;
  }
}
```

```

}

.tarifs {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: var(--gap);
}

@media (max-width: 600px) {
  .tarifs {
    grid-template-columns: 1fr;
  }
}

```

Si tu mets une image hero : `object-fit: cover`, `alt` utile, poids raisonnable.

Contenu minimum

Hero : une phrase nette ("Apprends X en une journée"). Programme : trois titres courts. Tarifs : "Solo" et "Duo" (ou equivalent) avec prix et liste courte. Formulaire : nom, email, choix de date (`select`), bouton.

Criteres de reussite

On comprend l'offre en cinq secondes.

La page ne casse pas en etroit.

Les ancrs du menu amènent aux sections.

Le formulaire est utilisable au clavier.

Changer `--couleur-principale` change clairement la marque.

Piege a eviter

Trop de sections "nice to have" (temoignages, logos partenaires, FAQ, Instagram...) avant d'avoir le socle. Reste sur le brief. Autre piege : deux boutons dans le hero de meme poids visuel. Un CTA principal suffit.

Bonus

Ajoute un voile de contraste sur le hero si texte sur image. Ou une pastille "Places limitees" en CSS simple dans une carte tarif (sans absolute si tu peux ; sinon absolute dans la carte en `position: relative`, ok).

A toi

Livre `landing.html` + `landing.css`. Chronometre 2 heures max pour la premiere version. Ensuite 20 minutes de polish (contrastes, gaps, textes). C'est valide quand tu oseais l'envoyer a un ami pour avis.

Chapitre 17 - Dark mode avec variables

Tu as déjà theme une page via `:root`. Le dark mode, c'est la meme idee, branchee sur la preference systeme (et/ou un choix manuel). Pas besoin de doubler tout ton CSS. Tu redefinis les variables.

Chez DanielCraft, un mode sombre lisible, c'est un confort nocturne, pas un filtre gris illisible. Contraste d'abord.

Prefers-color-scheme

Le systeme de l'utilisateur peut demander un theme sombre.

```
:root {
  --fond: #f7f5f0;
  --texte: #1b1b1b;
  --carte: #ffffff;
  --couleur-principale: #1a5f4a;
  --bordure: #ddd;
}

@media (prefers-color-scheme: dark) {
  :root {
    --fond: #121a17;
    --texte: #e8f0eb;
    --carte: #1a2822;
    --couleur-principale: #5dcaa5;
    --bordure: #2c3d34;
  }
}

body {
  background: var(--fond);
  color: var(--texte);
}
```

Tout ce qui consomme les variables bascule. Les classes de composants ne bougent pas. C'est exactement pour ca qu'on a discipline le theme a l'atelier variables.

Color-scheme (detail utile)

```
:root {
  color-scheme: light dark;
}
```

Ca aide le navigateur sur les controles natifs (scrollbars, champs) pour qu'ils ne restent pas "blancs agressifs" en theme sombre. Pas magique partout, mais bon reflexe.

Bascule manuelle (classe)

Le systeme ne suffit pas toujours : certains veulent forcer clair/sombre. Une classe sur `html` ou `body` :

```
html.theme-dark {
  --fond: #121a17;
  --texte: #e8f0eb;
  --carte: #1a2822;
  --couleur-principale: #5dcaa5;
  --bordure: #2c3d34;
}
```

Un bouton "Mode sombre" en JavaScript bascule la classe. Ce livre ne force pas le JS : tu peux tester en collant la classe a la main. Si tu ajoutes le JS plus tard, le CSS est deja pret.

Attention a la priorite : decide si la classe manuelle doit gagner sur `prefers-color-scheme` . Souvent oui (choix utilisateur explicite).

Images et ombres

Une ombre douce en clair devient un halo bizarre ou invisible en sombre. Variables :

```
:root {
  --ombre: 0 4px 14px rgba(0, 0, 0, 0.08);
}

html.theme-dark {
  --ombre: 0 4px 18px rgba(0, 0, 0, 0.45);
}
```

Les images photos restent des photos. Parfois tu baisses legerement l'opacite d'une image decorative en sombre, mais n'assombris pas une image informative jusqu'a la rendre inutile.

Formulaires en sombre

```
input,
textarea,
select {
  background: var(--carte);
  color: var(--texte);
  border: 1px solid var(--bordure);
}
```

Sans ca, des champs blancs eclatent dans une page sombre. Inclue-les dans le systeme de variables.

Contrastes : double check

Un vert menthe sur fond sombre peut etre magnifique et illegible en petit texte. Verifie :

Texte courant.

Liens.

Boutons (texte sur fond principal).

Labels et mentions.

Si besoin, eclaircis `--texte` ou fonce `--fond` . Le "vrai noir" `#000` pur sur OLED est discutable ; un presque noir teinte (`#121a17`) est souvent plus doux.

Landing : hero en sombre

Si le hero a déjà un voile sombre pour du texte blanc, il peut rester valable dans les deux thèmes. Si le hero dépend du `--fond`, teste les deux. Un CTA doit rester évidemment cliquable.

Erreur classique

Dupliquer toutes les classes (`.carte-dark`, `.bouton-dark` ...) au lieu des variables. Ou inverser seulement le fond et oublier les cartes/champs. Ou contrastes insuffisants "parce que c'est style".

Autre piège : dark mode active alors que ta page a encore des hex en dur partout. Corrige d'abord la discipline variables.

En vrai

Reprends le mini-projet ou l'atelier variables. Ajoute le media `prefers-color-scheme: dark` qui redefinit les variables. Bascule le thème OS (ou l'émulation dans les outils développeur : `prefers-color-scheme`). Regarde formulaire, cartes, header.

Ajoute ensuite `html.theme-dark` comme override manuel et teste les deux.

A toi

Active le dark mode par variables sur une page complète (pas un seul bloc). Checklist : body, cartes, bordures, bouton, inputs, ombres. Note une couleur que tu as du corriger pour le contraste. C'est bon signe : tu regardes vraiment.

Chapitre 18 - Perf CSS legeres : aller vite sans obsession

La performance, ce n'est pas seulement JavaScript et images. Le CSS peut aussi ralentir ou faire "sauter" une page. Bonne nouvelle : pour un site vitrine simple, quelques hygiene habits suffisent. Pas besoin de devenir ingenieur navigateur.

Chez DanielCraft, on vise "snappy et propre" : chargement raisonnable, peu de surprise visuelle, CSS maintenable.

Moins de CSS inutile

Chaque regle que tu n'utilises plus reste un poids mental (et parfois reseau). Quand tu termines un atelier, supprime les essais commentes sur vingt lignes. Garde un fichier clair.

Evite aussi les selecteurs ultra longs. Ils sont fragiles et le navigateur prefere des classes simples sur les elements concernees.

```
/* Fragile */
body div.main section.cartes article.carte p.prix span {}

/* Clair */
.carte-prix {}
```

Images : le vrai levier

Souvent, le CSS n'est pas le coupable n°1 : ce sont les images. Comprime. Redimensionne a la taille utile. `width/height` ou `aspect-ratio` pour reserver l'espace (moins de layout shift). `srcset` quand tu as plusieurs fichiers.

Une landing avec un hero 3 Mo restera lente quoi que tu fasses en CSS.

Eviter les reflows dramatiques

Animer `width`, `height`, `top`, `left` peut couter plus cher qu'animer `transform` et `opacity`. Pour les hovers de cartes, tu le sais deja : `translateY` + ombre.

Ne recalcule pas toute la page avec des mesures exotiques a chaque hover. Reste simple.

Fonts

Charger cinq familles Google Fonts pour trois titres ralentit. Une ou deux familles suffisent. `font-display: swap` (quand tu charges une webfont) evite le texte invisible trop longtemps.

Sur beaucoup de pages de ce livre, une stack systeme soignee ou une serif locale reste honnete et rapide.

Critical CSS ? Pas obligatoire ici

Les pros inlinent parfois le CSS critique du dessus de page. Pour ton niveau et un petit site, une feuille externe claire + images legeres + peu de polices, c'est deja excellent. N'optimise pas ce que tu n'as pas mesure.

Mesurer un peu

Ouvre l'onglet Réseau : poids total, poids images. Lighthouse / insights : regarde surtout les gros écarts (image énorme, contraste, etc.). Ne cours pas après 100/100 pour une page d'atelier. Cours après "rien d'énorme et bête".

CSS qui aide la stabilité

```
img,
video {
  max-width: 100%;
  height: auto;
}

.carte img {
  aspect-ratio: 4 / 3;
  object-fit: cover;
}
```

Reserver l'espace limite les sauts quand l'image arrive. Tes visiteurs te remercient inconsciemment.

Variables et maintenance

Un theme en variables, c'est aussi de la perf humaine : tu modifies moins de lignes, tu introduis moins de bugs, tu livres plus vite. La perf, ce n'est pas que des millisecondes.

Erreur classique

Ajouter une lib CSS entiere pour deux boutons. Ou dix animations permanentes. Ou des ombres a dix couches sur chaque carte. Ou charger la meme image hero en fond CSS et en balise `img`.

Autre piege : micro-optimiser le CSS pendant que le PNG du logo fait 2 Mo.

En vrai

Sur ta landing, liste les fichiers (CSS, images, polices). Note le plus lourd. Attaque celui-la. Recompresse l'image. Supprime une font inutile. Relance. Sens la difference.

Regarde aussi le nombre de regles CSS mortes : cherche une classe dans le HTML ; si elle n'existe plus, vire-la du CSS.

A toi

Fais un "pass perf" de 30 minutes sur une page : (1) image la plus lourde recompressée, (2) CSS nettoye d'essais, (3) verification `max-width` sur media, (4) une seule famille de police principale. Ecris le poids avant/apres de l'image. Objectif : -50 % sur cette image, ou mieux.

Chapitre 19 - Debug CSS : outils, outline, hypotheses

Ca ne s'affiche pas comme tu veux. Normal. Le debug CSS est un metier de detective : hypothese, test, observation. Pas de panique aleatoire ni de `!important` en rafale.

Chez DanielCraft, on debug comme on range un atelier : une cause a la fois, de la lumiere partout.

L'inspecteur, ton meilleur ami

F12 (ou clic droit → Inspecter). Selectionne l'element. Onglet Styles : regles actives, regles barrees. Onglet Calcule / Computed : valeur finale. Boite (margin/padding) visualisee.

Si une couleur "ne change pas", regarde qui gagne (specificite, ordre, inline). Tu as le chapitre cascade pour ca : l'inspecteur le montre en vrai.

Outline sur tout (temporaire)

Quand tu ne vois plus qui est qui :

```
* {
  outline: 1px solid tomato;
}
```

Ou plus cible :

```
.layout > * {
  outline: 2px dashed #1a5f4a;
}
```

Tu vois les boites. Tu vois les debordements. Tu enlèves ensuite. C'est un projecteur, pas une deco.

Certains navigateurs ont aussi un mode "montrer les grilles Flex/Grid" dans l'inspecteur : active-le sur le conteneur.

Hypothese claire

Avant de taper dix changements, ecris (meme mentalement) :

"Je pense que la carte debord parce que l'image n'a pas `max-width: 100%`."

Puis teste uniquement ca. Si faux, nouvelle hypothese : "le parent n'a pas de largeur bornee" / "un padding s'ajoute a une width 100%" / etc.

Changer cinq trucs a la fois, c'est tirer a l'aveugle.

Pieges frequents (checklist)

`max-width` manquant sur image ou iframe.

Parent en Grid/Flex mais enfant avec largeur fixe trop grande.

`overflow` cache quelque chose d'important (focus, ombre, menu).

Mauvaise `grid-area` (typo dans le nom).

Media query jamais atteinte (breakpoint trop bas/haut, ou faute de syntaxe).

Classe mal orthographiée dans le HTML (`.carte` vs `cart`).

Fichier CSS non lié, ou mauvais chemin, ou ancien cache (rechargement forcé).

Box model et width

```
* {  
  box-sizing: border-box;  
}
```

Souvent en début de feuille. Sinon `width: 100%` + padding peut faire déborder. Si un élément "dépasse un peu", vérifie le box-sizing.

Debug Grid

Colorie les zones. Vérifie que chaque ligne de `grid-template-areas` a le même nombre de cellules. Vérifie les noms `grid-area`. Regarde si un enfant sans area se place dans la première case libre et pousse le reste.

Debug Flex

`flex-wrap` oublié sur un menu qui débord. `min-width: auto` sur un enfant qui refuse de retrecir (parfois `min-width: 0` sur un enfant Flex/Grid aide dans des cas de texte long). Commence simple avant ce cas avancé.

Debug focus / a11y

Si le focus "disparaît", cherche `outline: none` ou un overflow qui coupe. Tab en regardant l'inspecteur (élément `:focus`).

Méthode en 5 minutes

1. Reproduire le bug (largeur précise).
2. Inspecter l'élément fautif.
3. Lire la propriété concernée (qui gagne ?).
4. Poser une hypothèse unique.
5. Tester. Si ok, nettoyer les outlines et commentaires de debug.

Erreur classique

Ajouter des marges négatives au hasard pour "ramener" un bloc. Ça masque le vrai problème. Ou créer une classe `.fix` avec `!important` sur tout. Ou modifier le HTML structure pour compenser un CSS confus sans comprendre.

En vrai

Casse volontairement une page qui marche : retire `max-width` sur une image, faute une `grid-area`, baisse un breakpoint. Puis debug avec la méthode. Remets propre. Tu entraînes le réflexe.

Fais la même chose sur un conflit de couleur (classe vs type). Regarde la règle barrée.

A toi

Prends un bug reel sur ton mini-projet (meme petit). Ecris l'hypothese en une phrase. Corrige. Ajoute en commentaire CSS une ligne `/ fix: image max-width - debord mobile /` le temps de t'en souvenir, puis efface le commentaire si trop bruyant. Le muscle, c'est l'hypothese, pas le commentaire.

Quiz final

Pas de piège. Relis une fois si besoin. L'idée : vérifier que tu peux avancer sans paniquer.

Questions

1. 1. Pourquoi préférer `header`, `main`, `nav`, `footer` à une pile de `div` anonymes ?

- A) Parce que ça change automatiquement les couleurs
- B) Parce que ça donne un sens clair à la structure (humains et machines)
- C) Parce que Flexbox refuse les `div`

1. 2. À spécificité égale, quelle règle gagne en général ?

- A) La première du fichier
- B) La dernière du fichier
- C) Celle écrite en anglais

1. 3. Où déclare-t-on souvent les variables CSS globales ?

- A) Dans `:root`
- B) Uniquement dans chaque balise `<img>`
- C) Dans le fichier `robots.txt`

1. 4. `display: grid` sert surtout à :

- A) Remplacer HTML par CSS
- B) Poser un layout en lignes et colonnes
- C) Compresser les images

1. 5. Dans `grid-template-areas`, chaque ligne du dessin doit :

- A) Avoir le même nombre de cellules
- B) Commencer par un `#`
- C) Être vide

1. 6. Pour un menu horizontal logo + liens, le réflexe le plus naturel est souvent :

- A) Flexbox
- B) Une table HTML à 12 colonnes
- C) `position: fixed` sur chaque lien

1. 7. Pour des vignettes produit de même taille, un bon duo est :

- A) `object-fit: cover` dans un cadre fixe / `aspect-ratio`
- B) `object-fit: fill` toujours, même si ça écrase
- C) Largeur fixe 2000px sur mobile

1. 8. Un vrai `label` de formulaire, c'est surtout :

- A) Un placeholder qui disparaît

- B) Un texte relie au champ (souvent via `for` / `id`)
- C) Une image decorative

1. 9. Ou placer en general la propriete `transition` pour un aller-retour doux ?

- A) Uniquement dans `:hover`
- B) Sur l'etat de base (ex: `.bouton`)
- C) Dans la balise `<title>`

1. 10. `:focus-visible` aide surtout a :

- A) Masquer le focus pour toujours
- B) Montrer clairement le focus clavier sans bruler tout le design
- C) Activer le mode sombre

1. 11. Pour un dark mode maintenable, le meilleur levier est :

- A) Dupliquer toutes les classes en `.dark`
- B) Redefinir les variables CSS (media et/ou classe)
- C) Mettre `filter: invert(1)` sur `body` et prier

1. 12. Bonne pratique generale :

- A) `!important` partout pour aller plus vite
- B) Variables + HTML semantique + test clavier regulier
- C) Tout positionner en `absolute`

Corriges

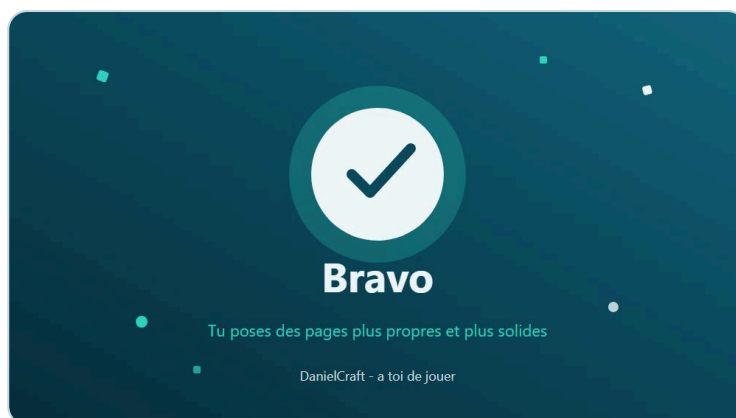
1-B, 2-B, 3-A, 4-B, 5-A, 6-A, 7-A, 8-B, 9-B, 10-B, 11-B, 12-B.

Si tu as 9/12 ou plus : tu es pret pour des pages reelles plus propres. Sinon, relis les chapitres lies (semantique, cascade, variables, Grid, images, formulaires, a11y, dark mode), sans dramatiser.

Mini debrief

Entoure deux questions ratees. Rouvre le chapitre correspondant. Refais l'exercice "A toi" de ce chapitre. Le quiz n'est pas un examen : c'est une carte des trous a boucher.

Bravo



Bravo. Tu sais poser des pages plus solides.

Sérieusement : bravo.

Tu as fini HTML et CSS - La suite. Pas juste survole. Tu as vu la sémantique, la cascade, les variables, Grid (bases et mise en page), Flex vs Grid, les images modernes, les formulaires stylés, les transitions légères, l'accessibilité suite, un mini-projet, des ateliers, le dark mode, un peu de perf, et du debug méthodique.

C'est exactement comme ça que ça marche : lire, essayer, planter un peu, corriger, continuer.

Ce que tu sais faire maintenant

Tu sais poser une page avec un vrai plan (Grid) et une vraie cohérence de marque (variables). Tu comprends mieux qui gagne en CSS. Tu choisis Flex ou Grid avec intention. Tu cadres des images sans les massacrer. Tu styles un formulaire pour qu'on ait envie de l'utiliser. Tu ajoutes du mouvement sans transformer le site en fête foraine. Tu penses clavier, contraste, labels. Tu bascules un thème sombre sans dupliquer tout le CSS. Tu debugues avec des hypothèses, pas avec de la panique.

Les bases du premier livre sont toujours là. La suite, c'est elles en mieux rangées, plus solides, plus accueillantes.

Petite mission (si tu veux)

Reprends ta landing ou ta page produit. Améliore juste trois trucs :

1. un thème cohérent 100 % variables (clair + sombre)
2. une grille de cartes en `auto-fit` + `minmax`
3. un passage Tab sans accroc + focus visible

Montre-la à quelqu'un. Demande ce qui est clair en cinq secondes. Écoute. Ajuste.

La suite, quand tu seras chaud

JavaScript, pour rendre la page un peu vivante (formulaires qui vérifient, menus, contenus qui se chargent...). Ou un vrai petit site multi-pages en ligne. Ou le livre JavaScript chez DanielCraft, si tu veux un chemin guide.

Mais pas ce soir si tu es fatigué. Repose-toi. Tu l'as mérité.

Encore bravo.

Et a bientôt pour la suite.

Tu as les briques. Maintenant, construis.